SEN381 - MILESTONE 3: Controlled Construction, Integration, Quality & Release Readiness

PED Version 3.0

Team Members:

Aidan Smith - 601819
Enrique de Sousa - 603110
Michael-John Robinson - 602310

1) PROBLEM & BUSINESS NEED / STAKEHOLDER ANALYSIS
- 1.1) Problem and business need
- 1.2) Stakeholder analysis
- 1.3) Linked requirement examples

2) SCOPE BASELINE & CONSTRAINTS
- 2.1) Scope baseline
- 2.2) Constraints
- 2.3) Constraint interaction

3) REQUIREMENTS & ACCEPTANCE CRITERIA
- 3.1) Functional requirements
- 3.2) Non-functional requirements
- 3.3) Prioritisation note

4) REQUIREMENTS TRACEABILITY MATRIX (RTM)
- 4.1) RTM EVOLUTION (M3)

5) ASRs AND QUALITY DRIVERS

6) RISK REGISTER & FORWARD ENGINEERING CONSIDERATIONS
- 6.1) Risk Register
- 6.2) Forward Engineering Considerations

7) ARCHITECTURE DECISION AND DIAGRAMS

8) DATA AND PERSISTENCE BASELINE
- 8.1) Entities, ownership and lifecycle
- 8.2) Initial data model
- 8.3) Persistence model, SPOF and recovery
- 8.4) Transactions, validation and concurrency

9) TECHNOLOGY-STACK DECISION

10) INITIAL DESIGN DECISIONS

11) INITIAL API / INTEGRATION DECISIONS

12) DEPLOYMENT COMPATIBILITY

13) M2 BASELINE CONFORMANCE

14) CONTROLLED CHANGES

15) CONSTRUCTION / CONFIGURATION

16) CI & AUTOMATED VERIFICATION

17) QUALITY STRATEGY APPLIED

18) BLACK-BOX / INTEGRATION / SYSTEM VALIDATION

19) PERFORMANCE / RELIABILITY

20) DEFECTS & QUALITY INTERPRETATION

21) RELEASE READINESS

22) TECHNICAL DEBT REGISTER

23) GITHUB GOVERNANCE, TEAM ACCOUNTABILITY & AI USAGE REGISTER
- 23.1) Repository governance evidence

23.2) Team working agreement
23.3) AI Usage Register
24) ENGINEERING DECISION LOG
25) PED QUALITY, DOCUMENT CONTROL & BASELINE SIGN-OFF
25.1) Document control
25.2) Baseline sign-off
26) REFERENCES

1) PROBLEM & BUSINESS NEED / STAKEHOLDER ANALYSIS

1.1) Problem and business need

The organisation currently has a disjointed approach to service requests. They are made by email, on WhatsApp, on spreadsheets, and on paper. This results in issues of duplicated and lost requests, lack of clarity over ownership, lack of visibility for requesters, inconsistent prioritisation, and poor accountability. Management is also less effective in getting reliable information on outstanding, overdue and completed requests, and sensitive information may be handled insecurely through informal channels. The proposed solution is a controlled digital environment that will ensure that there is one single record for every service request with a single and traceable record for all service requests from submission to closure. This will help the business to create value by enhancing the efficiency of operations, transparency, accountability and information management. Requests would be more visible to the requester, staff would have a better sense of ownership and prioritisation, management would have more accurate information for them to monitor and make informed decisions. The solution should be usable, secure, maintainable and financially sustainable to the organisation and deliver these benefits.

1.2) Stakeholder analysis

Stakeholder: Requesters
Requesters, people from the community who place a “service request”, need to have a convenient and easy to use way to report an issue and get an acknowledgement that their request was received. They also want to see the status of the request, the staff member assigned to the request, progress, delays and final resolution. They are primarily interested in timely and effective service, and have a relatively low influence on the system compared to management and staff. There can be a conflict if the elements are expecting a quick reply or a priority to be given to the request, while the staff and management is prioritising the requests according to the urgency, resources and organisation policies.

Stakeholder: Service Staff

Service staff should be responsible for receiving, reviewing, assigning, prioritising and resolving service requests. They want a system where they can access information around outstanding work, priorities, deadlines, request details and ownership. They have strong influence and interest in the system as they will be using the system regularly and their work will be directly affected by the system. Staff might not want a more flexible and readily used system while management may want more control, documentation and reporting. Some conflict may also occur if the system is seen as adding to administration or as involving monitoring of staff performance.

Stakeholder: Management
Management needs reliable, real-time information about the number and types of requests, workloads, overdue issues, resolution times and overall service performance. Their involvement in the project is great as they make decisions related to priorities, resources, policies and system adoption. Management has a keen interest in accountability, reporting and operational efficiency. However, their expectations may be different from those of management when management requires a high level of tracking and reporting and staff may feel that they are being monitored too much with extra administrative requirements or lack flexibility in the way that they work.

Stakeholder: IT/System Administrators
IT staff or system administrators require a solution that is secure, maintainable, scalable and manageable within the technical resources of the organisation. They handle user access, system configuration, data protection, backup and troubleshooting, and may even be responsible for integration with existing systems. They have a strong influence since technical choices can have long-term consequences for the sustainability of the platform. A potential conflict exists between IT's need for security, governance and controlled changes and the organisation's desire for a solution that is inexpensive, simple and quick to implement.

Stakeholder: Organisation/Business Owners
Business owners or senior decision makers in the organisation are interested in improvements that are measurable and not an ongoing financial or operational burden. They must have the solution that will meet the goals of the organisation to implement and sustain. They are powerful in that they are able to approve requirements, budgets, and major changes. The business value of their expectation of a cost-effective solution that can be relatively straightforward may clash with IT or management needs for more sophisticated security, reporting, integration and functionality which necessitates a balance between value and complexity and cost of the project team.

1.3) Linked requirement examples

An example of a requirement that traces directly back to one of the aforementioned stakeholder needs is that the requester must be able to see if the request has been received, assigned, delayed, resolved or closed. Thus, the system should enable the requesters to track

the current status of their requests during the request lifecycle. This requirement goes directly to the current situation where the requester doesn't know what is happening, and they have to reach out to staff in other ways to find out.

2) SCOPE BASELINE & CONSTRAINTS

2.1) Scope baseline

In scope:

- Submitting, categorising, viewing, and tracking service requests.
- Viewing request history and receiving feedback on request status changes.
- Allowing authorised staff to search, filter, view, assign, update, and resolve service requests.
- Recording staff actions, comments, and resolution information.
- Providing management with service activity information including open, overdue, resolved, and closed requests.
- Reporting on requests by category, status, and other justified dimensions.
- Supporting accountability and service performance analysis.

Out of scope:

- Features that are not directly related to the core service-request process.
- Additional functionality that cannot be justified by clear stakeholder value.
- Features that would introduce unnecessary scope, security, cost, schedule, quality or maintenance risks.

Deferred / future scope:

- Additional features proposed by stakeholders that require further analysis before implementation.
- Advanced functionality that cannot be completed within the initial project scope without compromising quality.
- Enhancements identified after the core system has been successfully implemented and tested.

In defense of advanced reporting and analysis being deferred, the primary emphasis of the initial system must be on the core service-request process. At this stage it would be difficult to justify the complexity of introducing more advanced dashboards and performance analytics since they are not necessarily a vital part of the minimum business requirements. The feature may be re-enabled when the core system is stable enough and enough data about the service-request system is gathered. It may be rethought if management determines that a more detailed performance evaluation is warranted and the extra development time, effort, cost, security and testing requirements can be justified within the project.

2.2) Constraints

Constraint type: Scope

- Description: The system should be targeted towards the minimum business capabilities: submission of service requests, tracking, assignment, and resolution of service requests. Additional features need to be warranted and not unnecessarily added to the project.
- Engineering implication: The team has to focus on core functionality and scope creep has to be avoided. Requirements should be baselined as early as possible with further features postponed if there is insufficient time or money.

Constraint type: Schedule

- Description: The project will have a fixed project deadline (19 October 2026) for when it will need to be designed, implemented and tested.
- Engineering implication: Essential requirements should be prioritised for development. The team should work incrementally and utilise the provided milestones so that a system is developed and working by the deadline.

Constraint type: Cost/Resources

- Description: The project has limited resources in terms of team members, development time, available technology and possibly limited access to paid services or infrastructure.
- Engineering implication: It is important to the team to adhere to the use of technologies and services that are readily available and to stay away from the need of using external technologies or expensive infrastructure. The features to be included should be based on the value that they add to development effort.

Constraint type: Quality

- Description: The system needs to be reliable, usable and maintainable while correctly handling service requests and status changes. A larger system should not be used if it does not yield an adequate or thoroughly tested solution.
- Engineering implication: Requirements must be testable and adequate time must be provided for the integration, validation and user testing. The team should strive for a smaller, consistent system rather than a bigger system that has defects.

Constraint type: Security

- Description: Sensitive information may be included in a service request and only be available to authorised users. Various users, such as requesters, staff and management could need different levels of access.
- Engineering implication: The system must implement authentication and role-based access control, protect stored and transmitted data, validate user input and prevent unauthorised access. Security must be built in from the beginning of the design process.

2.3) Constraint interaction

One of the major trade-offs is the relationship between schedule, scope and quality. If there is a time constraint on the project and a lot of extra features are added onto, the development will take more time and there will be less time to test and fix any defects. This may lead to lower overall quality and reliability of the system. In order to control this, the team must focus on the features and functions that are essential to the business and postpone any features that are not in order to give sufficient time to test the core system and ensure that the core is completed to a high standard.

3) REQUIREMENTS & ACCEPTANCE CRITERIA

3.1) Functional requirements

ID	Stakeholder	Priority	Acceptance criteria
FR-001: Submit a new informative service request	Requester - Member requesting service	High	Requester able to submit requests including fields. The system rejects requests without mandatory information. The system generates a unique ID.
FR-002: Categorise requests	Requester - Member requesting service	High	Requests have controlled category/s which are mandatory. The category is filterable
FR-003: View status of submitted requests	Requester - Member informed on status	High	Requesters can view the current status (e.g. Submitted, Assigned, In Progress, Resolved, Closed) of any request they submitted, from a request list or detail view.
FR-004: A history/list of previous submitted requests	Requester - Member interested in previous requests	Medium	Previous requests are viewable with the relevant information such as resolved, unresolved, etc.
FR-005: Receive meaningful feedback when status is updated	Requester - Member informed on request	High	The requester receives feedback about the request when it has been accepted, rejected, completed, etc. Staff are able to provide feedback
FR-006:	Staff - Member viewing requests	High	Staff can only view requests assigned to their role/department; attempting to open a request outside

ID	Stakeholder	Priority	Acceptance criteria
View service requests relevant to authorised staff			their role returns access denied or is not listed.
FR-007: Search, filter or sort requests	Staff - Member viewing requests	Medium	Staff are able to scan through requests based on criteria such as role, ID, service type, etc
FR-008: View full request details	Staff - Member viewing requests	High	Staff can view requester details, category, description, submission date, status history, and any attached comments/actions for a given request.
FR-009 Assign/accept responsibility for request	Staff - Member taking request	High	Staff can be assigned a request and also accept a request. Requests cannot be double assigned. Staff are able to see what requests they are responsible for
FR-010 Update request status	Staff - Member doing request	High	Staff is able to change the status of the request based on the status flow. The status must be updated on the requester's side too.
FR-011: Record relevant details on request resolution process	Staff - Member doing request	Medium	Staff are able to record the progression of the request resolution. Details must be stored for future references and learning.
FR-012: Resolve/close requests where authorised	Staff - Member doing request	High	Closed requests are moved to a history/archive view but remain retrievable for audit purposes; they are removed from the active/open requests list.
FR-013: View service activity information	Management - Member managing staff activity	High	Management is able to see the progression of requests being done by staff such as activity and completeness.
FR-014: Identify the status of requests	Management - Member viewing progress of requests	High	Management is able to see the status of requests, whether it's been assigned, opened, resolved, overdue, etc.
FR-015:	Management -	Medium	Management can filter requests by

ID	Stakeholder	Priority	Acceptance criteria
View request info based on criteria	Member viewing requests		category, status, etc, to see certain information
FR-016: Access information to support accountability and service-performanc e analysis	Management - Member monitoring staff performance	High	Management is able to see who is assigned to requests, the requests progress and the status of the request in terms of performance and quality.

3.2) Non-functional requirements

ID	Stakeholder	Priority	Acceptance criteria
NFR-001: Response time (Performance)	All users - Members submitting/viewing requests	High	95% of standard actions (submit, view status, search) complete within 2 seconds under normal load (up to 50 concurrent users).
NFR-002: Authenticated access (Security)	Staff/Management - Members accessing controlled functions	High	100% of staff/management functions require valid login. Unauthenticated access attempts are rejected and logged.
NFR-003: Role-based access control (Security)	Staff - Members with restricted role access	High	Staff cannot view or act on requests outside their authorised role/category. Verified through access-control testing with no exceptions.
NFR-004: Data protection (Security)	Requester - Member submitting personal/request information	High	All data in transit uses HTTPS/TLS. Passwords/credentials are hashed and never stored in plaintext. Verified via security review.
NFR-005: System availability (Reliability)	All users - Members relying on system access	Medium	The system maintains at least 99% uptime during defined operational hours. Downtime is logged with cause recorded.

NFR-006 Browser compatibility	All users - Members accessing via standard browsers	Low	The system is functional on at least two current major browsers (e.g. Chrome, Edge/Firefox) at common screen sizes.
NFR-007 Ease of use (Usability)	Requester/Staff - Member with no prior training	Medium	A first-time user can submit a request or update a status within 3 attempts without external help. Verified via usability test (3-5 participants).
NFR-008 Data integrity on failure (Reliability)	All users - Members relying on submitted data being preserved	Medium	A backup/recovery mechanism exists. Simulated failure/restore test confirms no submitted request data is lost.
NFR-009 Maintainability	Development team - Members maintaining/extending the system	Medium	Code follows an agreed structure/style standard, includes documentation for key modules, and passes static analysis with no critical issues.
NFR-010 Status change traceability (Auditability)	Management - Member requiring accountability evidence	High	Every status change is logged with user ID, timestamp, and previous/new status. 100% retrievable via audit/history view.

3.3) Prioritisation note

Requirements were rated **High** where they are essential to the core business need identified in the Project Brief, providing a reliable, traceable way to submit, manage, monitor and report on service requests, or where their omission would create a direct security or accountability gap.
Requirements were rated **Medium** where they materially improve usability, oversight, or operational quality, but are not strictly required for the system to operate in a controlled and functional manner.
Requirements were rated **Low** where they provide additional convenience or polish without affecting core capability, business value, or risk exposure.

4) REQUIREMENTS TRACEABILITY MATRIX (RTM)

Req ID & requirement	Source / priority	ASR / quality driver	Architecture responsibility	Data / persistence impact	Design / interface decision	Technology decision	Implementation evidence	Verification evidence	Status	ADR / change / risk ref.

FR-001 Submit service request	Requester / High	NFR-004, NFR-001, NFR-010	Requests module	ServiceRequest + first StatusHistory row; mandatory fields NOT NULL; unique ReferenceNo	Single synchronous transaction	ASP.NET Core 8 MVC; EF Core + SQL Server	PR #3, #16, #21, #23: RequestService.SubmitAsync (request + first StatusHistory in one transaction), unique ReferenceNo, M3_CoreSchema migration	Unit, black-box EP/BVA, integration, E2E – all pass (§27)	Implemented & verified	R-005
FR-002 Categorise requests	Requester / High	–	Requests module	Category reference table; mandatory FK	Controlled lookup + dropdown; server-side category check	ASP.NET Core 8 MVC; EF Core + SQL Server	PR #3 (Category lookup) + server-side active-category check (PR #23)	Unit, black-box EP, integration – pass	Implemented & verified	D-001
FR-003 View status	Requester / High	NFR-001, NFR-003	Requests module	CurrentStatus on ServiceRequest; query scoped to requester	State pattern	ASP.NET Core 8 MVC; EF Core + SQL Server	MyRequests + Details with requester scoping (PR #23)	Unit, integration, E2E – pass	Implemented & verified	R-003
FR-004 Request history	Requester / Medium	NFR-001	Requests module	No hard deletes; closed requests retained	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	MyRequests lists all own requests incl. closed (PR #23)	Unit + E2E; history split not asserted	Implemented – verification partial	FEC #4
FR-005 Feedback on status update	Requester, Staff / High	NFR-010	Assignment → Notification (in-process)	RequestComment (Feedback, visible to requester)	In-process INotificationService, same transaction	ASP.NET Core 8 MVC; EF Core + SQL Server	InProcessNotificationService, same transaction (PR #19, #25)	Unit + E2E staff journey – pass	Implemented & verified	D-003
FR-006 Staff see authorised requests only	Staff / High	NFR-003, NFR-002	Users/Auth + Security modules	StaffCategoryScope filters every staff query	Single authorisation point in service layer	Cookie auth + own User table + policy authorisation (CC-001)	StaffCategoryScope filter in AuthorisationService / search (PR #18, #21)	Unit, black-box decision table, integration – pass	Implemented & verified	R-002; FEC #1
FR-007 Search / filter / sort	Staff / Medium	NFR-001	Requests module	Indexes on status, category, assignee, date	DI repositories –	ASP.NET Core 8 MVC; EF Core + SQL Server	RequestSearchCriteria + RequestRepository.SearchAsync (PR #21, #25)	Unit (scope, closed filter); text/sort not automated	Implemented – verification partial	FEC #5
FR-008 Full request details	Staff / High	NFR-003, NFR-004	Requests module	Joins request, requester, history, comments	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	StaffRequests/Details + GetDetails (PR #23, #25)	Unit + integration access tests – pass	Implemented & verified	R-002
FR-009 Assign / accept	Staff / High	NFR-010, NFR-001	Assignment module (RequestWorkflowService)	Single AssignedStaffId; optimistic concurrency (RowVersion)	Optimistic locking	ASP.NET Core 8 MVC; EF Core + SQL Server	AssignAsync/AcceptAsync, RowVersion (PR #20, #25)	Unit + integration concurrent accept – pass; MC-02 pass	Implemented & verified	D-002; R-003

FR-010 Update status	Staff / High	NFR-010, NFR-003	Assignment module (RequestWorkflowService)	Status update + StatusHistory in one transaction	State pattern – D-005; feedback via	ASP.NET Core 8 MVC; EF Core + SQL Server	State pattern + TransitionGuard + CHECK constraint (PR #16, #20)	Black-box state transition (25 pairs), unit, E2E – pass; MC-01 pass	Implemented & verified	R-003
FR-011 Record resolution details	Staff / Medium	NFR-010	Assignment module (RequestWorkflowService)	RequestComment (ResolutionNote), append-only	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Mandatory ResolutionNote, staff-only notes (PR #25)	Unit + E2E – pass	Implemented & verified	FEC #4
FR-012 Resolve / close	Staff / High	NFR-003, NFR-010	Assignment module (RequestWorkflowService)	Status → Closed; archived, never deleted	State pattern	ASP.NET Core 8 MVC; EF Core + SQL Server	Close transition; IncludeClosed filter; no deletes (PR #25)	Unit + E2E – pass	Implemented & verified	R-002; R-003
FR-013 Service activity info	Management / High	NFR-010	Reporting (read-only queries)	Read-only queries over requests + history	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	ReportingService + Management/Dashboard (PR #27)	Integration + manual MV-01 – pass	Implemented & verified	FEC #6
FR-014 Status incl. overdue	Management / High	NFR-001	Reporting (read-only queries)	"Overdue" needs a due-date rule – not yet defined	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Status counts on dashboard (PR #27); overdue deferred (CC-002)	Integration counts + MV-01 – pass	Partial	R-003
FR-015 Filter by criteria	Management / Medium	NFR-001	Reporting (read-only queries)	Reuses FR-007 indexes	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Management/Requests filters (PR #27, CC-003)	Service-level unit test only	Implemented – verification partial	FEC #5
FR-016 Accountability info	Management / High	NFR-010	Reporting (read-only queries)	Depends on complete StatusHistory	DI repositories	ASP.NET Core 8 MVC; EF Core + SQL Server	Staff-activity table (PR #27)	Integration + MV-01	Partial (CC-003)	FEC #6
NFR-001 Response time	All / High	ASR	Cross-cutting	Indexing; single DB is bottleneck	Modular monolith, in-process calls	ASP.NET Core 8 MVC; EF Core + SQL Server	Indexes + paging; load test (PR #29)	Manual load test – §24	Verified (test env only)	FEC #5
NFR-002 Authentication	Staff/Mgmt / High	Security	Users/Auth + Security modules	User credentials; failed logins logged	IPasswordHasher / Identity	Cookie auth + own User table + policy authorisation (CC-001)	Cookie auth, policies, anti-forgery, rejection logging (PR #18, #22, #30)	Integration + unit – pass; log line manual	Implemented & verified	R-002
NFR-003 RBAC	Staff / High	ASR	Users/Auth + Security modules	Role + StaffCategory Scope	Single authorisation point in service layer	Cookie auth + own User table + policy authorisation (CC-001)	Policies + IAuthorisationService in every service (PR #17, #18)	15 tagged test methods + MV-02 – pass	Implemented & verified	R-002; FEC #1
NFR-004 Data protection	Requester / High	ASR	Users/Auth + Security modules	PasswordHash only; personal data sensitive	IPasswordHasher	Cookie auth + own User table + policy authorisation (CC-001)	PBKDF2 220k iterations, HTTPS/HSTS, Secure cookie (PR #18, #30)	PasswordHasherTests – pass; TLS by review	Implemented & verified	R-002

NFR-005 Availability	All / Medium	Reliability	Azure App Service	Single DB is a SPOF	No external dependency in the status path	Azure App Service F1	No monitoring in M3 scope	Not executed	Not verified	R-004; R-005
NFR-006 Browser compatibility	All / Low	–	MVC Razor views	None	Razor MVC views	Razor views + Bootstrap	Razor + Bootstrap	Manual (Chrome only)	Partial	FEC #3
NFR-007 Ease of use	Requester/ Staff / Medium	–	MVC Razor views	None	Razor MVC views	Razor views + Bootstrap	Razor views	Usability test not run	Not verified	–
NFR-008 Data integrity on failure	All / Medium	Reliability	Data-access layer	Backup/restore baseline	Backup before each migration	ASP.NET Core 8 MVC; EF Core + SQL Server	Backup-before-migration procedure (§26)	Restore test not run	Not verified	D-004; R-005
NFR-009 Maintainability	Dev team / Medium	Quality	Cross-cutting	Version-controlled EF migrations	DI	ASP.NET Core 8 MVC; EF Core + SQL Server	Layering, format gate, 2-approval review (PR #15)	CI format step + CodeQL 0 alerts	Verified	R-001; FEC #2
NFR-010 Status traceability	Management / High	ASR	Audit module	Append-only StatusHistory, same transaction as change	Single synchronous transaction	ASP.NET Core 8 MVC; EF Core + SQL Server	AuditService StatusHistory in same transaction (PR #19, #25)	Unit + integration – pass; MC-03 pass	Implemented & verified	D-003; D-004; R-003

4.1) RTM EVOLUTION (M3)

The Section 4 RTM was updated in place (implementation, verification and status columns; CC-001 noted for the security rows). The table below extends it to the M3 chain. Since M2: 16/16 FRs implemented; NFR-001/002/003/004/009/010 verified. Test files are named in §21/§23; "CI" = run #55.

Req	Acceptance criterion	Implementation	Tests	Evidence	Status
FR-001	Mandatory info; unique ID	RequestService.SubmitAsync, NextReferenceNo, unique index	Unit, BB EP/BVA, TE-05, E2E	CI; MC-04	Verified (TD-001 open)
FR-002	Category from controlled list	Category lookup + server check	Unit, TE-02	CI	Verified
FR-003	Requester sees own status	MyRequests, Details, requester scoping	Unit, TE-08, TE-10	CI	Verified
FR-004	History of own requests	MyRequests incl. closed	Unit, E2E	CI	Partial verification
FR-005	Feedback on each change	NotificationService, same transaction	Unit, TE-11	CI; MC-03	Verified (in-app only, TD-008)

Req	Acceptance criterion	Implementation	Tests	Evidence	Status
FR-006	Staff see own categories only	StaffCategoryScope, AuthorisationService	Unit, TE-04, TE-08	CI; MV-03	Verified
FR-007	Search / filter / sort	RequestSearchCriteria, SearchAsync	Unit (scope, closed filter)	CI	Partial – text/sort not automated
FR-008	Full details; others denied	Details + GetDetails	Unit, TE-08	CI	Verified
FR-009	Never double-assigned	Accept/Assign, RowVersion	Unit, TE-07	CI; MC-02	Verified
FR-010	Status follows the flow	State pattern, TransitionGuard, CHECK	BB TE-03, unit, TE-11	CI; MC-01	Verified
FR-011	Resolution recorded	Mandatory ResolutionNote	Unit, TE-11	CI	Verified
FR-012	Closed archived, retrievable	Close + IncludeClosed, no deletes	Unit, TE-11	CI	Verified
FR-013	Management sees activity	ReportingService, Dashboard	TE-09	CI; MV-01	Verified
FR-014	Status incl. overdue	Status counts	TE-09	CI; MV-01	Partial – overdue (CC-002)
FR-015	Management filters	Management/Requests	Unit (service level)	CI	Partial – no HTTP test (CC-003)
FR-016	Accountability info	Staff-activity table	TE-09	CI; MV-01	Partial – no quality metrics
NFR-001	p95 ≤ 2 s, 50 users	Indexes, paging	Load test (§24)	§24	Verified (test env only)
NFR-002	Login required; rejections logged	Cookie auth, policies, anti-forgery	Integration, unit, TE-06	CI	Verified
NFR-003	No access outside role/category	Policies + IAuthorisationService	15 tests incl. TE-04, TE-09	CI; MV-02	Verified
NFR-004	HTTPS; hashed passwords	PBKDF2 220k, HSTS, Secure cookie	PasswordHasherTests	CI	Verified (TLS by review)
NFR-005	99% uptime	–	–	–	Not verified
NFR-006	Two browsers	Razor + Bootstrap	Manual (Chrome)	MV-01/02	Verified
NFR-007	Usable in 3 attempts	–	–	–	Verified
NFR-008	No data lost on restore	Backup procedure (§26)	–	–	Verified
NFR-009	Style + static analysis	Format gate, review, CODEOWNERS	CI format, CodeQL	CI	Verified

Req	Acceptance criterion	Implementation	Tests	Evidence	Status
NFR-010	Every change logged	AuditService, same transaction	Unit, TE-05, TE-11	CI; MC-03	Verified

Every High-priority FR except FR-014 and FR-016 has passing evidence; those two are partial for documented scope reasons, not defects.

5) ASRs AND QUALITY DRIVERS

ASR candidate: NFR-003 Role-based access control (Security, High priority)
- Already flagged in M1 Forward Engineering Consideration #1 (RBAC) as influencing "architecture and technology-stack selection"
- Measurable/testable expectation (from M1): Staff cannot view/act on requests outside authorised role/category, verified through access-control testing with no exceptions
- How it influences architecture/data/technology/design: RBAC necessitates architecture and technology that provides for centralised authentication and authorisation. Data model and design should restrict access to roles and categories at the back end/API level.

ASR candidate: NFR-001 Response time (Performance, High priority)
- Measurable/testable expectation (from M1): 95% of standard actions complete within 2 seconds under normal load (up to 50 concurrent users)
- Connects to Forward Engineering Consideration #5 (Scalability under concurrent use)
- How it influences architecture/data/technology/design: The architecture needs to be scalable, with optimised data access and services, to ensure it can handle the processing and concurrent users efficiently.

ASR candidate: NFR-010 Status change traceability (Auditability, High priority)
- Measurable/testable expectation (from M1): Every status change logged with user ID, timestamp, previous/new status; 100% retrievable via audit/history view
- How it influences architecture/data/technology/design: Needs an audit-log data structure and backend mechanisms to capture all the status changes with whom, when and the history of status.

ASR candidate: NFR-004 Data protection (Security, High priority)
- Measurable/testable expectation (from M1): HTTPS/TLS in transit, hashed credentials, verified via security review
- How it influences architecture/data/technology/design: Needs HTTPS/TLS, secure password hashing, and security conscious technology and architecture decisions to secure data in transit and at rest.

6) RISK REGISTER & FORWARD ENGINEERING CONSIDERATIONS

6.1) Risk Register

Organisations building e-service systems frequently struggle with poor risk practices, particularly under budget and schedule pressure. This gap is a recurring contributor to project failure (Pilliang & Munawar, 2022). Vague and generic risk entries provide little actionable value whereas effective registers have a clear cause with measurable impact and an owner who can act on it.

Risk ID	Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
R-001	Two-reviewer PR approval becomes a bottleneck given only 3 team members, especially near deadlines when availability is uneven	Small team size means every PR needs buy-in from both remaining members, who may be busy with their own sections	Medium	Medium	Medium	Agree a team working-agreement turnaround time for reviews stagger individual workstreams so PRs don't all land at once	If a reviewer is unavailable, escalate to the third member or delay merge rather than bypass the two-approval rule	Aidan Smith	Mitigated
R-002	Sensitive request data is handled without adequate access control, creating a security exposure	RBAC not yet designed; requirements may under-specify who can view what	Medium	High	High	Capture data-sensitivity and access-control needs explicitly as NFRs now	If a gap is found post-implementation, restrict the affected view immediately and treat as a formal defect	Michael-John Robinson	Mitigated
R-003	Core request-status workflow is under-specified, causing rework once management views are designed in M2	Status-transition rules not fully defined at requirements stage	Medium	Medium	Medium	Explicitly define allowed status transitions as part of FR acceptance criteria before workflow design in M2	If a missing transition is discovered later, treat as a controlled requirements change, not an ad hoc code fix	Michael-John Robinson	Mitigated – verified
R-004	A requirement or NFR quietly assumes deployment infrastructure the	Cost constraint not cross-checked against requirements as	Low	Medium	Medium	Research realistic free hosting options before finalising NFRs	If discovered late, scale back the specific requirement	Enrique de Sousa	Open, mitigation pending validati

	team cannot actually afford or access	they are written					rather than the whole architecture		on
R-005	Database single point of failure / data loss	All request and audit data sits in one database; an outage stops the service and loss or corruption destroys accountability evidence	Low	High	Medium	Daily off-host backups, a backup before each migration	Restore latest backup (RPO ≤ 24 h, RTO ≤ 4 working hours)	Michael -John Robinso n	Open
R-006	Login is not protected against repeated password guessing, and there is no password reset or email verification.	The authentication change uses cookie authentication with our own user table instead of the full ASP.NET Core Identity store, which would have provided lockout, reset and two-factor out of the box.	Medium	Medium	Medium	Generic error message for every failure, slow PBKDF2 hashing, failed attempts logged with the account name, self-registration limited to the Requester role.	If suspicious attempts appear in the logs, disable the affected account and add throttling or a lockout before further use.	Aidan Smith	Open
R-007	A green CI run could give false confidence because database-backed integration tests and the performance test are not part of the blocking gate.	The CI runner has no SQL Server, and the load test needs a deployed instance.	Medium	Medium	Medium	The release evidence table lists skipped and manually-run tests separately, so only executed tests count as evidence.	If a manual run was not recorded, treat that behaviour as unverified and report it as a residual risk.	Aidan Smith	Open – mitigate d by recorded manual run
R-008	Broken or unformatted code reaching main	Occurred twice while checks were optional.	High	Medium	Medium	Build-test + dependency-audit made required status checks; the gate then blocked at the PR	Build-test + dependency-audit remain required status checks	Aidan Smith	Mitigate d

6.2) Forward Engineering Considerations

Design choices are often reasonable at the moment they're made but measurably reduce a system's maintainability and scalability as the system grows if the implications are not considered early (Verdecchia, Kruchten, Lago, & Malavolta, 2021).

1. Authentication & role-based access control (RBAC) [Addressed: Section 7/9]

- Why it matters now: Three distinct user types need different visibility and permissions, shaping how requirements and data models are framed.
- Later Influences: Architecture and technology-stack selection.
- Missing Information: Exact permission boundaries per role
- Postponement Risk: Retrofitting access control after implementation typically requires touching most of the codebase.

2. Automated testing & CI [Deferred to M3]

- Why it matters now: If requirements aren't written in testable, acceptance-criteria form now, retrofitting testability later is costly.
- Later Influences: CI pipeline design and test strategy.
- Missing Information: Which requirements are highest-risk and therefore need the most test coverage.
- Postponement Risk: Requirements without measurable acceptance criteria are difficult to verify automatically, forcing manual testing.

3. Deployment environment & platform compatibility [Addressed: Section 12]

- Why it matters now: Cost/schedule constraints mean the team should understand hosting limitations before committing to requirements that assume unavailable infrastructure.
- Later Influences: Technology-stack decision and staging deployment.
- Missing Information: What free/low-cost hosting options are actually viable for the stack choices.
- Postponement Risk: A requirement or NFR could quietly assume infrastructure the team cannot deploy affordably.

4. Data backup & recovery [Addressed: Section 8]

- Why it matters now: Service-request records are the core asset of the system. Loss of data has direct consequences for every stakeholder group.
- Later Influences: Data/persistence design and operational readiness review.
- Missing Information: What recovery point/time expectations are realistic.
- Postponement Risk: No backup strategy discovered only after a data-loss incident, rather than factored in from the start.

5. Scalability under concurrent use [Addressed: Section 5/7]

- Why it matters now: Staff and management views involve simultaneous access to shared request data.
- Later Influences: Architecture and performance testing.
- Missing Information: Realistic expected concurrent user counts.
- Postponement Risk: Performance NFRs left vague now become impossible to test meaningfully.

6. Observability & monitoring [Deferred to M3]

- Why it matters now: Management's oversight capabilities depend on the system being able to report on its own state.
- Later Influences: Operational readiness and incident response.
- Missing Information: What minimum monitoring is realistic to implement within constraints.
- Postponement Risk: It will become hard to know if the system is degrading without planned logging

7) ARCHITECTURE DECISION AND DIAGRAMS

Realistic architecture alternatives considered:

- Alternative A: Modular monolith. One deployable backend exposing a REST API, organised internally into modules. A web front end serves all three user types, and a single relational database holds the data.

- Alternative B: Microservices. Separate services for authentication, request management, assignment, notifications and audit, each with its own data store, communicating via APIs or a message broker behind an API gateway.

- Alternative C: Serverless/Backend-as-a-service (BaaS). Managed authentication, a hosted database and cloud functions, with a thin client. Little custom server code is written.

- Alternative D: Separate applications per user type. Independent Requester, Staff and Management applications sharing a database or duplicating logic.

Selected architecture, justified against the ASRs above, M1 constraints, team capability, deployment implications, complexity:

Architecture Selected: Alternative A

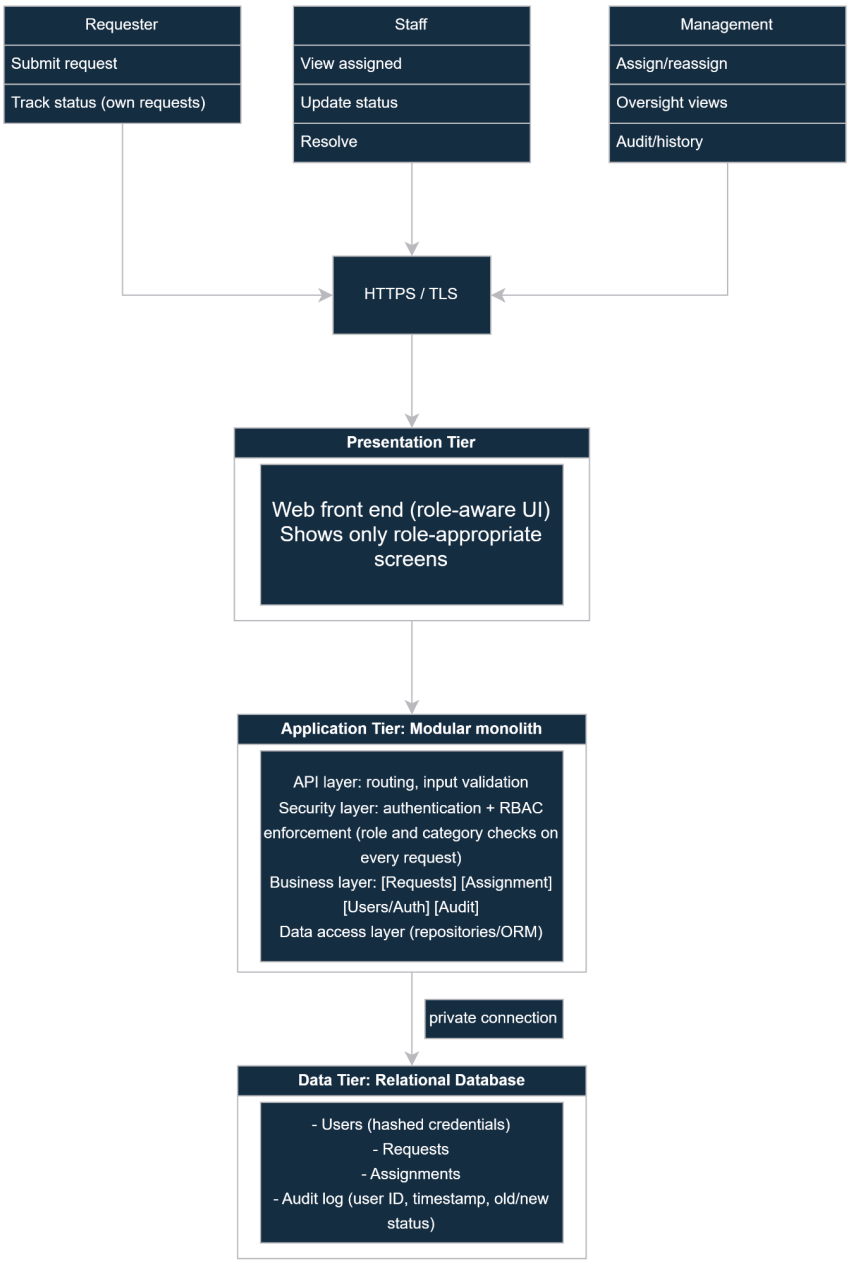
Justification against ASRs:

- NFR-003 (RBAC): It’s not just in the UI that one authentication and authorisation level is passed through at the API/service level. Role and request category access is checked first before any business logic or data excess is executed. This makes the M1

test (“no exceptions”) tractable, as there is a single enforcement point to test as opposed to multiple points.

- NFR-001 (Response time): With 50 concurrent users, a single well-indexed relational database and in-process module calls comfortably meet 2 seconds for 95% of actions. There are no hops across the network between services. If load increases, the stateless backend could be horizontally scaled behind a load balancer without redesign, thus avoiding the distribution cost in the meantime and solving the Forward Engineering Consideration #5.
- NFR-010 (Traceability): Status changes and their audit records are written in the same database transaction. The audit view is a simple query, and a change has to be logged in order to be successful. For a distributed design, cross-service consistency would have to be solved to ensure this.

Architecture diagram(s) - responsibilities/boundaries for the three user types (Requester/Staff/Management) and interactions:



Architecture vs. technology vs. logical layers vs. physical deployment tiers - explicitly distinguished:

- Architecture: How is the system organised into parts, and how do they communicate? This is the structural decision driven by the ASRs.
- Technology: Which concrete tools implement the architecture? These are interchangeable choices made after the architecture.
- Logical Layers: How is the code separated by responsibility? These are conceptual and exist within the software, regardless of where it runs.
- Physical Development Tiers: Where does the software run, on which machines or processes?

8) DATA AND PERSISTENCE BASELINE

8.1) Entities, ownership and lifecycle

Entities are derived from FR-001 to FR-016. The owner is the only responsibility area allowed to change that data; others read it through the owner.

Entity	Owner	Lifecycle
ServiceRequest	Request Management (status/assignment changed by Workflow)	Submitted → Assigned → In Progress → Resolved → Closed; never deleted (FR-012)
User / Role	Identity & Access	Users deactivated, never deleted, so audit references stay valid
Category	Request Management	Controlled list; retired categories deactivated
StaffCategoryScope	Identity & Access	Defines which categories each staff member may see (FR-006, NFR-003)
StatusHistory	Workflow & Assignment	Append-only audit trail (NFR-010)
RequestComment	Workflow & Assignment	Append-only feedback and resolution notes (FR-005, FR-011)

Requester contact details are personal information; no retention period is set yet, so this is raised with stakeholders (POPIA).

8.2) Initial data model

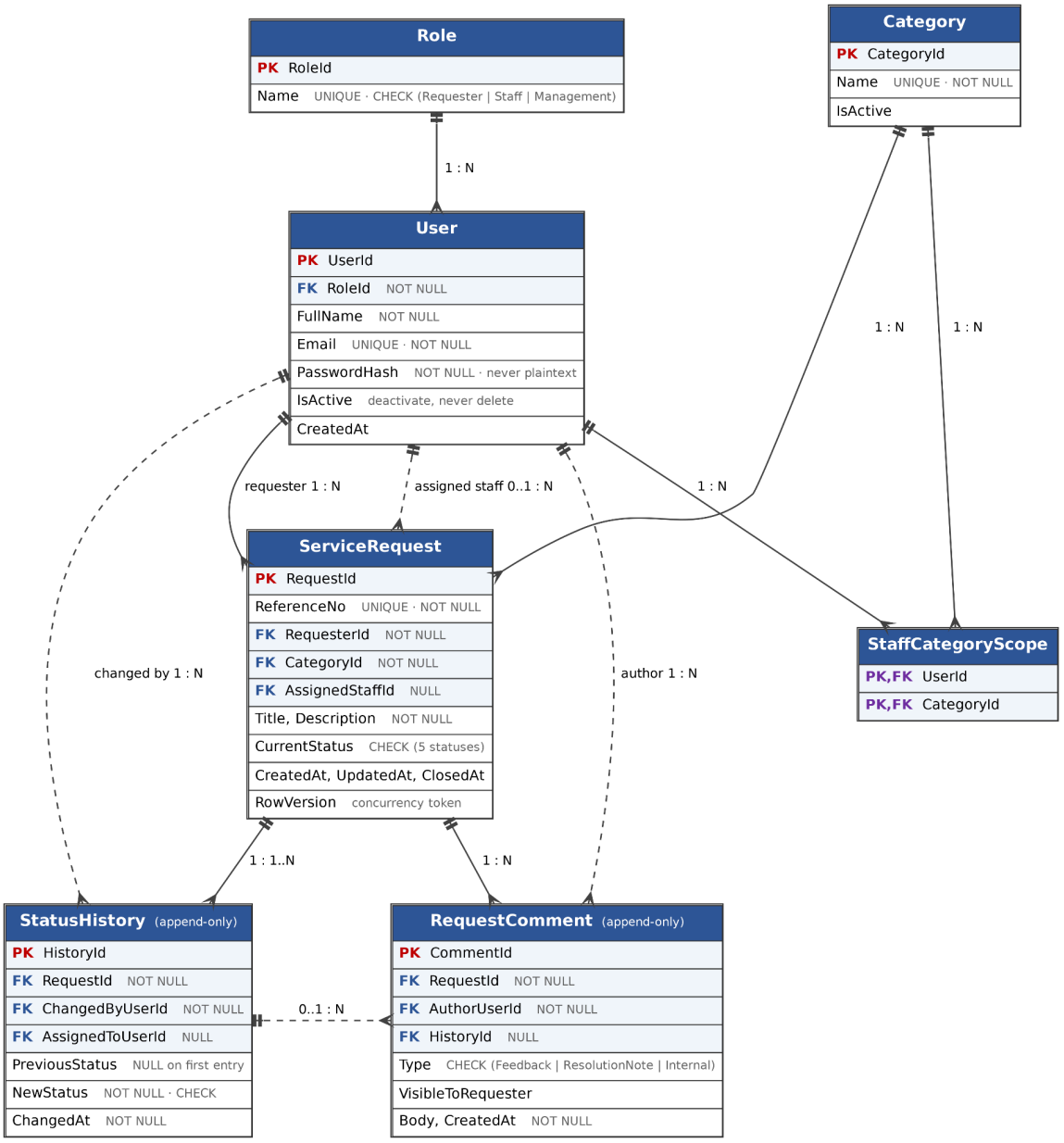


Figure 13.1 – CivicConnect initial data model

Key integrity rules: NOT NULL on mandatory request fields and a unique ReferenceNo (FR-001); FK to Category (FR-002); CHECK on the five status values (FR-003); allowed transitions enforced in the service layer; only PasswordHash stored (NFR-004).

8.3) Persistence model, SPOF and recovery

A single relational database is selected. The data is structured and strongly related, status changes and history must commit together, and management needs cross-request filtering and counts – all natural in SQL. This fits the C# decision: assignment 1 section 3 scored ASP.NET Core with Entity Framework Core (3.5 vs 3.0 for MERN), which gives migrations, transactions and concurrency tokens.

The database is a single point of failure and the main bottleneck (NFR-001, NFR-005). At the expected load (≤ 50 concurrent users) this is accepted: indexes, connection pooling and paging come first; caching and replicas are not justified without load-test evidence, and permissions must never be served stale. Recovery (NFR-008): automated daily backups stored off the host, plus a backup before every schema migration as recommended in Assignment 3 section 3.4, and a restore test before M4. Proposed targets: RPO ≤ 24 h, RTO ≤ 4 working hours.

8.4) Transactions, validation and concurrency

Assignment 2 section 2 compared a single synchronous transaction with an asynchronous audit write for request creation and recommended the synchronous transaction, because the async option leaves a window where a request has no audit entry (NFR-010). Applied here:
- Request creation (FR-001) and status change (FR-010) write the request and its StatusHistory row in one transaction; the service layer owns the boundary.
- Validation in three layers (Assignment 2 section 2.2): UI for usability, service layer for business rules, database constraints as the safety net.
- Assignment (FR-009) uses optimistic locking with a RowVersion check, as Assignment 2 section 2.2 recommends for this low-contention case.

9) TECHNOLOGY-STACK DECISION

Component	Decision	Evidence
Backend	ASP.NET Core (C#) .NET8	LTS releases get 3 years of support vs. 2 for STS; .NET 8 is supported until Nov 2026, the longest remaining window of any released version
Frontend	Razor Pages / MVC	Built into ASP.NET Core, no separate framework/licensing decision needed
Persistence	Entity Framework Core + SQL Server	EF Core's change-tracking supports the NFR-010 audit-trail requirement without a custom mechanism
RBAC	ASP.NET Core Identity + policy-based authorization	Built-in, directly satisfies NFR-003 without a third-party RBAC library
Licensing	MIT	Not yet independently confirmed, verify against the LICENSE file before finalizing ADR
Dependency Risk	Lower than MERN/npm, not zero	NuGet's core packages are more centrally Microsoft-maintained than

		npm's
Cost	Fully free at every layer	SDK, EF Core, SQL Server Express/SQLite, Visual Studio Community, GitHub/Actions, no paid tier required to build or run CivicConnect

10) INITIAL DESIGN DECISIONS

Design problem 1: Tightly coupled request-processing responsibilities
- A2 evidence/alternatives considered: A2 Task 1.2 compared State Pattern (groups behaviour by request state; better cohesion, but may lead to over-engineering if the workflow is simple) and Strategy Pattern (interchangeable processing rules; only useful if request types require different processing rules). A2 Task 1.3 suggested State if the issue is the status workflow and Strategy if multiple processing rules are required.
- Final M2 pattern/approach selected: State Pattern, maintained as a lightweight pattern. Each RequestState contains a RequestState, which specifies its allowed transitions, and guards. A single RequestWorkflowService processes the guard, applies the change and writes the audit record in one transaction. Strategy is not implemented.
- Why it fits CivicConnect requirements/architecture/constraints: The problem is the status workflow, as indicated in the brief. All categories have exactly one lifecycle (category is data, not behaviour), and so Strategy does not have any justification. There are role/category rules enforced in the service layer (NFR-003) and changes in status are audited in one place, the workflow service (NFR-010). The small design adapts to the scope, schedule and quality constraints, with approximately six states. It is located within the monolith, and does not have an impact on deployment.
- Expected benefit (coupling/cohesion/extensibility/maintainability/testability): Higher cohesion (SRP), no status conditionals in Request, easy extension by adding a state (OCP), transition rules in one place, and isolated unit testing of valid, invalid and unauthorised transitions.
- Complexity/trade-off introduced: Added about 6 small classes and a service rather than one status field. Modest learning curve and an over-engineering risk if the workflow doesn't change. Mitigation: Do not use a shared transition table, and do not use a generic workflow engine.
- Affected modules/components/classes/interfaces: Requests, Assignment, Audit and Security modules; Request, RequestState and its subclasses; RequestWorkflowService, TransitionGuard; IRequestRepository, IAuditLogRepository.
- If the M2 choice differs from the A2 recommendation: CivicConnect-specific evidence that changed the judgement: The M2 choice does not differ from the A2 recommendation. The controlled transitions in the brief A2 favour State over Strategy, as do the NFR-003, NFR-010 and single shared lifecycle in M2.

Design problem 2: Business logic becoming dependent on infrastructure
- A2 evidence/alternatives considered: A2 Task 1.2 compared the Adapter Pattern (isolates a specific external system, maintainability benefits, but adds a layer) with Dependency Injection (dependencies supplied rather than created. Lower coupling and better testability, but too many interfaces burden a small app). A2 Task 1.3 recommended DI generally, with Adapter only for a specific external system.
- Final M2 pattern/approach selected: IRequestRepository, IUserRepository, IAuditLogRepository, IPasswordHasher, IClock, plus a unit-of-work abstraction. Implementations are wired in at start-up through the framework's DI container. Adapter is not adopted and is reserved for any external integration approved later.
- Why it fits CivicConnect requirements/architecture/constraints: It makes the layered rule (services depend downward on abstractions) enforceable. Scope has no external integrations, so Adapter has nothing to adapt. Hashing sits behind IPasswordHasher, so the algorithm can be upgraded without touching business logic (NFR-004). Injected security and audit services cannot be bypassed and are testable with fakes (NFR-003, NFR-010). Hosting or database may change because of cost or BC Desktop compatibility, and interfaces limit that impact.
- Expected benefit (coupling/cohesion/extensibility/maintainability/testability):
- Complexity/trade-off introduced: Extra interfaces and DI configuration (the risk A2 flagged). Mitigation: interfaces only at real boundaries (persistence, auth, time). Repository abstractions can leak ORM details. Swapping the database is cheaper but not free.
- Affected modules/components/classes/interfaces: Users/Auth, Requests, Assignment, Audit, Security and Data-access layers; the interfaces above; RequestWorkflowService, AuthenticationService, AuthorisationService, AuditService and their implementations, the composition root.
- If the M2 choice differs from the A2 recommendation: The M2 does not differ from the A2 recommendation. DI is adopted as A2 recommended. Adapter is consciously not taken because the approved scope has no external integrations. Revisit through change control if email or SMS is approved.

11) INITIAL API / INTEGRATION DECISIONS

Interaction: Status change → requester feedback. Evolves the Assignment 2 section 3 interaction (Status Management → Notification), updated for the approved scope.
- Responsibilities and information exchanged: Producer: RequestWorkflowService in the Assignment module, after a valid, authorised status transition. Consumer: INotificationService, which records a requester-visible Feedback entry (RequestComment) shown on the requester's request view (FR-003, FR-005). Data passed: RequestId, ReferenceNo, previous status, new status, changed-at, changed-by user, optional staff comment. No requester contact details are passed.
- Mechanism and why it is proportionate: A synchronous in-process call through the INotificationService interface, injected via DI and executed inside the same database transaction as the status update and StatusHistory row . This differs from Assignment 2 section 3, which recommended an asynchronous event because an external email/SMS provider could slow down or fail. The approved scope has no external provider, so that reason

no longer applies; in a modular monolith a queue and worker would add a network boundary and infrastructure the Azure F1 tier does not justify.
- Validation, failure, security and versioning: The workflow validates the transition and the user's role/category (NFR-003) before calling the interface. Because the feedback is written in the same transaction, a failure rolls back the whole status change – a requester can never miss an update (FR-005) and the audit trail stays complete (NFR-010). The call never leaves the process, so no credentials cross it; feedback is only returned by queries scoped to the owning requester. The interface is internal and versioned with the code. If email/SMS is approved later, an adapter can consume a RequestStatusChanged event asynchronously behind the same interface (Assignment 2 section 3 Mechanism B) without changing the workflow code.
- Web interface established so far: GET/POST /ServiceRequests/Create and GET /ServiceRequests/Details/{id} (ASP.NET Core MVC, anti-forgery token, server-side validation), merged in PR #3. The staff status-update endpoint is Planned.
- ADR / RTM: RTM rows FR-005, FR-010 and NFR-005 updated.
- Forward Engineering Consideration: External notifications (email/SMS) are deferred. If stakeholders approve them, they go through Appendix E change control, with a new risk for provider outage.

12) DEPLOYMENT COMPATIBILITY

Concern	Finding	Status/Action
Hosting platform	Azure App Service, F1 (free) tier	Up to 10 web apps, 1 GB memory, shared compute, no cost
Capacity vs. requirements	Comfortably covers NFR-001 for academic-project scale	Performance under real load still unverified
SLA	None on F1 tier	Acceptable for staging/demo
Secrets/config	Azure App Service Application Settings	Use instead of committing connection strings/credentials to source
R-004 status	Open → Open, mitigated pending validation	Free-hosting option identified and matched to stack
Deferred	CI/CD pipeline to Azure; production-tier upgrade decision	Not required until M3

13) M2 BASELINE CONFORMANCE

Release candidate/build being assessed: release/m3-rc, commit 285017d, CI run #55 (build-test and dependency-audit green, 92/92 tests passed, 0 skipped). Classified CONDITIONALLY READY.

Main implemented components and how they relate to the M2 architecture/design:

M2 Element	Status	Evidence
Modular monolith, one deployable	Conformed	One ASP.NET Core 8 MVC app; modules for Requests, Assignment, Users/Auth, Security, Audit, Reporting
Logical layers	Conformed	Controllers -> Services -> Repositories -> DbContext (Section 20)
Single RBAC enforcement point	Conformed	[Authorize(Policy)] & IAuthorisationService in every service method
Status pattern + RequestWorkflowService	Conformed	State classes, TransitionGuard, CHECK constraint; all 25 status pairs tested
DI with repository, hasher and clock interfaces	Conformed	Registered in Composition/*; a test resolves every service
Relational DB (EF Core + SQL Server)	Conformed	Migrations including M3_CoreSchema; manual checks on LocalDB
Reporting (FR-013 to FR-016)	Conformed, with scope gaps	See CC-002 and CC-003

Persistence and API/integration decisions demonstrated in the running system:

- Request creation and status changes write the request, the StatusHistory row and the feedback comment in one transaction (TE-05, MC-03).
- Optimistic locking with RowVersion prevents double assignment (TE-07, MC-02).
- Validation happens in three layers: UI, service, and database constraints (MC-01 for the status CHECK, MC-04 for the unique ReferenceNo).
- The synchronous in-process INotificationService call matches Section 11 and ADR D-003.
- Anti-forgery, HTTPS and HSTS, and cookie flags are enforced (TE-06, DEF-006/PR #30).

Significant deviations from M2 (minor coding detail changes do NOT need an ADR - only real deviations):

Deviation 1: authentication (CC-001/D-005)

- Original decision: ASP.NET Core Identity with policy-based authorisation (Section 9).

- Evidence/problem: The project already owned User and StaffCategoryScope. IdentityDbContext would add a second user model, extra tables, a package and a second migration history.
- Revised decision: Cookie authentication, the framework PasswordHasher<T> behind IPasswordHasher, and your own User table with a Role column. Policy-based authorisation is kept.
- Consequence: There is no lockout, reset, email confirmation or 2FA (R-006, TD-002, TS-003). Hashing and the single authorisation point are unchanged.

Deviation 2 (scope-level, not architectural): overdue reporting (CC-002)

- Original decision: FR-014 includes an “overdue” state.
- Problem: no due-date rule was ever agreed with stakeholders.
- Revised decision: status counts only for M3.
- Consequence: FR-014 stays Partial, and a later fix needs one migration and one reporting query.

14) CONTROLLED CHANGES

ID	Original Decision	Trigger	Revised Decision	What stays the same	Consequences
CC-001	Chose ASP.NET Core Identity + policy-based authorization and put password hashing behind IPasswordHasher.	CivicConnect already owns its User and StaffCategoryScope entities. IdentityDbContext would add a second user model, extra tables, an extra package and a second set of migrations to coordinate with one migration owner.	Cookie authentication + the framework PasswordHasher<T> wrapped by IPasswordHasher + our own User table with a Role column + the planned policy-based authorisation.	Hashed credentials, one authorisation enforcement point, policy names, IPasswordHasher.	(a) No lockout, password reset, email confirmation or two-factor (b) One user model, one migration history. (c) Later move to Identity or hand-built features is possible without changing IPasswordHasher or the policies
CC-002	FR-014 requires management to see "overdue" requests (M1/M2 RTM: "Overdue needs a due-date rule – not yet defined").	No due-date or service-level rule was ever agreed with stakeholders, so "overdue" cannot be calculated without inventing a business rule.	Deliver FR-014 without the overdue state in M3: the dashboard shows counts for every status. Overdue is deferred until a rule (e.g. days per category) is approved.	Status counts, Management-only access, read-only reporting, the RTM row for FR-014.	FR-014 is Partial in the RTM (§27). Adding it later needs a due-date column or per-category SLA, one migration and a new reporting query – no change to the workflow.
CC-003	M3 ownership plan: FR-015/FR-016 deferred, ManagementController.Requests stubbed.	The search service built for staff (FR-007) already supported every management filter, so FR-015 cost one controller action and no new data access.	FR-015 implemented in M3 (PR #27): Management/Requests filters/sorts/pages all requests including closed. FR-016 delivered partially as the staff-activity table.	Read-only management access, the single search service, NFR-003 checks.	FR-015 Implemented; FR-016 Partial – performance/quality metrics (e.g. time to resolve) need a stakeholder decision. No extra risk: same RBAC path as staff search.

15) CONSTRUCTION / CONFIGURATION

Repository Strategy	
Topic	**Approach**
Governance	Protected main; every change by pull request; two approvals from non-authors; no self-approval; no admin bypass. CODEOWNERS now requests the right reviewers automatically but does not replace the two-approval rule.
Branches	Short-lived (merged within 2-3 days), named by purpose: feature/<area>, test/<area>, chore/<area>, release/<name>.
Exclusive ownership	Every file has one owner (Documentation/M3-Ownership-and-Branches.md, mirrored in .github/CODEOWNERS), so shared-file merge conflicts are designed out.
Frozen contracts	Program.cs, GlobalUsings.cs, .csproj, Common/*, Composition/* and Models change only through a "CONTRACT CHANGE" issue and a small lead-approved PR.
One migration owner	Only Michael creates EF Core migrations, so the model snapshot never conflicts.
Branches used	chore/m3-skeleton, chore/ci-quality-gate, feature/security-auth, feature/audit-notifications, feature/data-migration, feature/data-repositories, feature/workflow-states, feature/request-submission, feature/staff-workflow, feature/management-overview, test/fakes-and-security-unit, test/unit-workflow, test/blackbox-suite, test/integration-e2e, test/performance, release/m3-rc
Release candidate	m3-rc

Configurations and Secrets	
Area	**Approach**
Repository	No secrets committed. appsettings.json holds only the LocalDB connection string. Local overrides are in .gitignore.
Local development	The demo seed password is read from the key "SeedPassword" set with dotnet user-secrets, never hard-coded.
Staging	Connection string and other settings are Application Settings, which override appsettings.json.

CI	Needs no secrets, uses only the repository's built-in token.

What was Built	
Topic	**Build**
Layering	One dependency direction: Controllers -> Services -> Repositories -> DbContext. Controllers make one service call per action. Services never touch HttpContext or EF types, repositories hold queries only and never call SaveChanges. The service layer owns the transaction.
Errors	Services return Result/Result<T> with an ErrorCodes value. AppControllerBase.MapFailure turns codes into HTTP outcomes in one place. Only ConcurrencyConflictException crosses layers.
Cookies	HttpOnly, SameSite=Lax, 8-hour sliding expiry, Secure whenever the request is HTTPS (SameAsRequest); HTTPS enforced by redirection and HSTS outside Development.
Two fences	Authorize(Policy=...)] on controllers (outer, coarse) + IAuthorisationService inside every service method. Refusals use one message so existence is never revealed.
Login/Registration	Same generic message and similar work for unknown email, wrong password and inactive account, failures logged without the password and with line breaks stripped, self-registration can only create Requesters, only local return URLs followed.
Passwords	Only a salted PBKDF2-HMAC-SHA512 hash, 220,000 iterations.
Anti forgery	Every POST has [ValidateAntiForgeryToken]. Unauthenticated and forbidden attempts are rejected and logged with user id and path.
Auditing	AuditService only adds StatusHistory rows, InProcessNotificationService only adds RequestComment rows, neither saves. The workflow saves both with the status change in one transaction.
Dependency injection	Everything registered once in Composition/*, a test resolves every service and controller, so a missing registration fails a test, not production.
Testibility	ICurrentUser, IClock and in-memory fakes allow security and audit tests with no database or web server.

16) CI & AUTOMATED VERIFICATION

Every PR into main or release/** (and every push to them) runs .github/workflows/ci.yml on a clean ubuntu-latest runner:

- build-test – Release build → dotnet format --verify-no-changes → dotnet test (all categories except Performance) → TRX report uploaded as an artifact.
- dependency-audit – fails if any direct or transitive NuGet package has a known vulnerability.

CodeQL (C#, codeql.yml) runs on PRs and weekly; Dependabot opens weekly NuGet and Actions update PRs.

Check	Tool	Blocking?
Build	dotnet build -c Release	Yes – required check
Static / code quality	dotnet format (style + Roslyn/xUnit analyzers)	Yes – in build-test
Automated tests	dotnet test, 92 cases	Yes – in build-test
Dependencies	dotnet list package --vulnerable; Dependabot	Audit yes; Dependabot advisory
Code security	CodeQL	No – report-only (TD-006)

Quality gate: main needs two non-author approvals plus green build-test and dependency-audit on an up-to-date branch. The gate did its job in M3: it blocked PR #26 twice (DEF-003, DEF-004). Before the checks were required, two broken builds reached main (DEF-001, DEF-002).

Evidence: release-candidate run #55 (commit 285017d, release/m3-rc) – Success, 57 s; "Passed! Failed: 0, Passed: 92, Skipped: 0" – https://github.com/AidanSm2103/SEN381_CivicConnect/actions/runs/38878210477. Workflow: https://github.com/AidanSm2103/SEN381_CivicConnect/blob/main/.github/workflows/ci.yml

Test category	Min	Actual	Where
Unit / component	≥ 5	52	Unit/*.cs (services, states, auth, hashing)
Black-box functional	≥ 5	23	BlackBox/*.cs – EP, BVA, decision tables
API / integration	≥ 3	15	Integration/*.cs – real HTTP pipeline
E2E / system	≥ 2	2	E2E/*.cs – requester and staff journeys
Negative / unauthorised	≥ 3	45	Tagged Type=Negative
High-priority req./risk	≥ 5	81	Tagged Priority=High (R-002, R-003)
Black-box techniques	≥ 2	4	EP, BVA, decision table, state transition
Performance	1	1	Manual, outside CI (§24)

17) QUALITY STRATEGY APPLIED

Risk / ASR	Why it got stronger verification	How it was verified	Lighter verification (and why)
R-002 / NFR-003 RBAC, NFR-002 authentication (High)	A gap exposes other people's personal request data – the highest-impact failure in the register.	Two independent fences, each tested: unit (AuthorisationService), black-box decision table (role × action), integration (real cookies/redirects), manual MV-02. 15 NFR-003 and 5 NFR-002 test methods.	–
R-003 / FR-010 workflow, FR-009 double assignment (High)	An illegal transition or double assignment corrupts the audit trail and the staff workload.	State-transition coverage of all 25 status pairs; unit tests per state; concurrent-accept integration test; E2E lifecycle; manual RowVersion check MC-02.	–
NFR-010 traceability (High ASR)	Accountability evidence must never be partial.	Unit tests assert one history row + one feedback per change and nothing written on conflict; integration test checks the first history row; MC-03 on SQL Server.	–
FR-001/FR-002 submission (High)	Most used entry point; bad data here propagates everywhere.	EP + BVA on description and category; integration + E2E.	–
FR-013–FR-016 reporting (High/Medium)	Read-only, so failure is wrong numbers rather than data damage.	One integration test (counts + read-only) and manual MV-01.	Lower data risk; FR-014 overdue and FR-016 metrics not defined yet (CC-002/003).
NFR-001 performance	Separate concern; needs a running instance.	One manual load scenario (§24).	Outside the CI gate; residual risk at production scale.
NFR-005/007/008 availability, usability, restore (Medium)	–	Not executed in M3.	Need hosting, participants or a restore drill beyond the M3 time budget – carried as residual risk (§26).

Risk-based prioritisation reasoning: test effort was spent where a failure would be invisible to users but damaging (access leaks, silent audit gaps, double assignment) rather than where a failure is obvious and harmless (layout, read-only counts). This is why 81 of the 92 automated cases address High-priority requirements and 45 are negative cases – the risks in the register are about what must not happen.

18) BLACK-BOX / INTEGRATION / SYSTEM VALIDATION

Technique	Why it applies	Where
Equivalence partitioning	Inputs fall into groups: empty / whitespace / valid text; invalid / active / inactive / unknown category	SubmitRequestBlackBoxTests
Boundary value analysis	2,000-char description limit: 1,999 / 2,000 / 2,001	SubmitRequestBlackBoxTests
Decision table	Role × scope × action decides if a change is allowed	TransitionMatrixBlackBoxTests (RoleVsAction)
State transition	Workflow is a state machine; all 25 from/to pairs checked	TransitionMatrixBlackBoxTests; RequestStateTests
Negative testing	Top risks (R-002, R-003) are about what must not happen	45 negative cases

Expected results come from the §3 acceptance criteria. All automated rows passed in CI run #55 (92/92).

ID	Basis	Technique	Input → expected	Result	Interpretation
TE-01	FR-001 mandatory text; limit 2,000	EP + BVA	"", " ", valid; 1,999/2,000/2,001 chars → only valid and ≤ 2,000 accepted	Pass	Rule matches DB column (nvarchar(2000))
TE-02	FR-002 controlled category	EP; integration	Category 0 / 99 inactive / 1000 / 9999 → rejected, nothing saved	Pass	Enforced on server, not just the dropdown
TE-03	FR-010, R-003	State transition	All 25 status pairs → only the next step allowed	Pass	Full transition coverage
TE-04	NFR-003, FR-006, R-002	Decision table	Requester / staff in / out of scope / manager / anonymous assign → only in-scope staff succeeds	Pass	Service-layer fence works without the UI
TE-05	FR-001, NFR-010	Integration	Valid POST → 302 to Details; 1 request + 1 history row	Pass	Whole pipeline cooperates
TE-06	NFR-002	Integration, negative	POST without anti-forgery token → 400	Pass	CSRF protection active
TE-07	FR-009, R-003	Integration, negative	Two staff accept the same request → one wins, other gets conflict	Pass	No double assignment (in-memory DB; see MC-02)
TE-08	FR-006, FR-008	Integration, negative	Out-of-scope or other requester's request → not listed / AccessDenied	Pass	List and direct URL both protected
TE-09	FR-013, NFR-003	Integration	Manager sees dashboard but cannot act; staff → AccessDenied	Pass	Reporting is management-only and read-only
TE-10	FR-001/003/004	E2E	Register → submit → My Requests shows it as Submitted	Pass	Requester journey works end to end
TE-11	FR-005, FR-009–012, NFR-010	E2E	Accept → start → resolve (note required) → close; requester sees each update	Pass	Full lifecycle with feedback and archive

Manual checks on LocalDB (2026/10/16) – what the in-memory CI database cannot show (R-007):

ID	Req	Step	Result	Status
MC-01	FR-010	Set Status = 'Cancelled'	Rejected by CK_ServiceRequests_Status (Msg 547)	Pass
MC-02	FR-009	Accept the same request in two sessions	A accepted; B "already been assigned"; one assignee; RowVersion changed	Pass
MC-03	NFR-010, FR-005	Query request, history, comments after MC-02	One history row + one feedback, same transaction; refused click wrote nothing	Pass
MC-04	FR-001	Duplicate ReferenceNo	Rejected by unique index (Msg 2601)	Pass
MV-01/02	FR-013, NFR-003	Manager and staff open /Management/Dashboard	Manager sees counts; staff gets Access denied	Pass
MV-03	FR-006	staff2 opens Work queue	Only her categories listed	Pass

Screenshots of every manual check are kept as evidence. MC-02 used one staff account in two sessions because the seed data gives the two staff no shared category; two different staff are covered by TE-07.

Critical E2E/system journey(s) demonstrated:

1. Requester journey (TE-10; FR-001, FR-003, FR-004): register → log in → submit a request → see it listed as Submitted in My Requests.
2. Staff lifecycle journey (TE-11; FR-005, FR-009 to FR-012, NFR-010): accept → start → resolve (resolution note required) → close, with the requester seeing feedback at each step and the closed request archived but retrievable.

Both ran automatically in CI run #55 and were also run live against the local staging instance (Section 26). Together they cover the submit-to-closure lifecycle, which is the core business need in Section 1.1. Limits: browser-level UI behaviour isn't automated, and CI uses the in-memory database, so SQL Server behaviour is covered by the manual LocalDB checks (MC-01 to MC-04).

19) PERFORMANCE / RELIABILITY

Test Design	
Item	**Detail**
Operation	Request submission (FR-001), then viewing it (FR-003/FR-008) and listing own requests (FR-004). Most complete vertical slice; one transaction writes the request and its StatusHistory row.
Target	NFR-001: 95% of standard actions within 2 seconds at up to 50 concurrent users. Pass = p95 < 2,000 ms with zero errors; the test

	asserts this.
Workload	50 virtual users, each with its own cookie session. One login, then 5 repeats of: GET form (token), POST new request, GET details, GET "my requests". = 250 submits, 250 details, 250 lists (750 measured actions) + 50 logins. No think time. Logins are reported separately and excluded from the p95 because deliberate 220,000-iteration hashing happens once per session. PERF_USERS and PERF_ITERATIONS change the size.
Tool	Performance/SubmitRequestPerformanceTests.cs: drives a RUNNING instance over HTTP using only .NET. Excluded from CI (Category=Performance), skips unless PERF_BASE_URL, PERF_EMAIL and PERF_PASSWORD are set. Writes TestResults/perf-results.txt.

What tests do not prove	
Limitation	**Effect**
One operation family, one instance	Staff search on large data, the management overview and mixed workloads are untested.
Shared limited hardware	Numbers describe a student environment, not a municipal one.
No think time or user network latency	Figures show whether the design has an obvious bottleneck at the NFR-001 load; they are not predictions.
Only about 250 new rows	Query time on millions of rows is untested; the indexes on Status, RequesterId and ReferenceNo are a design expectation, not a measurement.
No sustained, failover or recovery testing	NFR-005 (99% uptime) and recovery targets (RPO <= 24 h, RTO <= 4 working hours, Section 8.3) are verified separately

20) DEFECTS & QUALITY INTERPRETATION

ID	Defect	Severity	Status	Fix	Regression evidence
DEF-001	main did not compile after PR #18 (RedirectToRoleHome missing)	High – all work blocked	Closed	Added in PR #22; build-test made a required check	Gate blocks any repeat
DEF-002	main did not compile: SecurityIntegrationTests used factory helpers that did not exist yet	High	Closed	CivicConnectFactory implemented – PR #26	6 security tests now pass in CI
DEF-003	Format gate failed on main (whitespace in 3 files)	Low (blocked CI)	Closed	dotnet format – PR #26	Format step green
DEF-004	9 analyzer warnings in black-box test skeletons failed the gate	Low (blocked CI)	Closed	Tests implemented – PR #26	23 black-box cases pass
DEF-005	user-secrets set-up failed (no UserSecretsId) – no demo users	Medium (dev only)	Closed	UserSecretsId added – PR #30	Manual: seeded logins work
DEF-006	Rejected access not logged; cookie flags unfinished (NFR-002)	Medium	Closed	Logging + SameSite/Secure – PR #30	Security tests pass; log checked manually

Static and security findings

Source	Finding	Handling
dotnet format	Whitespace + 9 analyzer warnings	Fixed (DEF-003/004)
CodeQL	0 alerts	Recorded; report-only until triaged (TD-006)
dependency-audit	0 vulnerable packages	Required check
Dependabot	4 Actions updates merged (#6–#9); 4 EF Core 8.0.32 patches open (#31–#34)	Merged after the RC so the tested build is not changed
Release review	No lockout; logging/cookie gaps	Gaps fixed (DEF-006); lockout = TD-002, residual risk

Interpretation: 92/92 passing shows the implemented behaviour holds after every change, not that SQL Server behaviour or the unverified NFRs (005, 007, 008) work. The suite is weighted to the top risks (45 negative, 81 high-priority cases). Six of seven defects were integration/process faults, and the gate only stopped them once it was enforced. "0 CodeQL alerts" means no known patterns, not "secure" – the lockout gap is a design issue tools cannot see.

21) RELEASE READINESS

Release Evidence

Area	Result	Interpretation	Concern/Action
Build & CI	Blocking checks: build-test and dependency-audit, both required on main with branches up to date. CodeQL: passed.	Code compiles and blocking checks pass on a clean machine; does not prove correct behaviour.	Main was merged broken twice before the checks were required, once for a build break and once for formatting. Fixed by making build-test a required check.
Automated regression	CI run: 92 run, 92 passed, 0 failed, 0 skipped. The split: unit 52, black-box 23, integration 15, E2E 2.	Covered behaviours still work after each change. Skipped tests are not evidence; list each with a reason.	No skipped tests, or list each with its reason. The in-memory provider doesn't check SQL constraints, real RowVersion or transactions. These are covered only by manual runs on LocalDB.
API/integration	All tests passed, unauthenticated and forbidden access redirected, DI resolves every service.	Real HTTP pipeline (routing, cookies, anti-forgery, DI, views) works together, on the in-memory database. Does not prove SQL Server behaviour (constraints, RowVersion, transactions).	SQL Server behaviour was verified only by a manual LocalDB run.. Action: add a SQL Server or SQLite container to CI after M3.
System/E2E	2 journeys passed: requester register, login, submit, view; staff accept, start, resolve, close with requester feedback. Also run live against local Staging	Main journeys work end to end for the implemented scope.	Covers the implemented scope only. Browser-level UI behaviour isn't automated. The staging run is local and single-machine.
Static/Security	Format check: passed. CodeQL: 0 alerts, severity none, handling fixed. Dependency audit: no vulnerable packages found	A clean scan means no KNOWN patterns or vulnerable packages, not the absence of design flaws (e.g. no lockout).	A clean scan doesn't mean the design is safe. Known gaps: no login lockout or throttling, open self-registration. Found in the release review: the cookie secure flag and access-denied logging were unfinished; both fixed in PR #30.
Performance	Operation:submit, view, list Environment: local, release build, local database. 50 Total users.	Meets NFR-001 for submit/view/list at 50 users in the test environment only.	Meets or misses in this environment only. A single machine, no network delay or think time, and few rows. Production scale is untested. Action: repeat on a hosted tier or a larger data set.
Known defects	7 defects recorded (DEF-001–DEF-007): 6 closed with regression evidence; 1 open, low severity (DEF-007, coverage collector missing) – not a blocker (§25).	Honest list of what is known to be wrong.	Anything high severity is a blocker. Low-severity items go in the residual risk table.

Acceptance	16/16 FRs implemented: 11 fully verified, 5 partial (FR-004, FR-007, FR-014 overdue, FR-015, FR-016). NFR-005/007/008 not verified (§27).	Maps evidence back to stakeholder requirements.	Each gap needs an owner and a date.

Classification Rule	
Current classification: CONDITIONALLY READY	
Class	**Condition**
READY	Build and all blocking checks green on the tagged commit; every Must requirement has passing evidence. No open high-severity defect or security finding; performance target met.
CONDITIONALLY READY	Must tier works and is evidenced, with named gaps that do not break core flows. Each condition needs an owner and a date.
NOT READY	Any Must requirement lacks passing evidence, a high-severity defect or security finding is open, or the release cannot be built and run from the tag.

Residual Risk Statement			
Residual Risk	**Why it remains**	**Mitigation**	**Next action**
Production-scale load and data volume untested	50 users, a few hundred rows, free-tier or local environment	Indexes on filter columns, paging, no obvious bottleneck at NFR-001 load	Repeat on a paid tier or larger data set
Brute-force password guessing not throttled	No lockout or rate limiting	Generic error, slow hashing, failed logins logged	Add lockout/rate limiting or move to Identity beyond a pilot
SQL Server behaviour not covered by CI	CI uses the in-memory database, which has no real transactions, CHECK constraint or RowVersion	Unit, pipeline, integration and E2E tests run in CI; SQL behaviour checked by a recorded manual run	Add a SQL Server service container or SQLite provider

Open self-registration	Stated scope choice; no email verification	Registration only creates Requesters, who see only their own requests	Decide on email verification with stakeholders

Rollback/Recovery	
Item	**Approach**
Artefact to restore	The previous M3 candidate. Rollback = redeploy that tag's build to the App Service.
Data concern	Schema changes are not safely reversible by migration once real data exists.
Procedure	Back up the database immediately before every migration. On a failed release: restore that backup, then redeploy the previous tag.
Configuration and cookies	Application Settings are independent of the build and stay. Cookies from the new build are rejected harmlessly by the old one, users log in again.

22) TECHNICAL DEBT REGISTER

Debt ID	Description	Why it was Accepted	Impact if unaddressed	Planned resolution
TD-001	The reference number is generated as "count + 1" inside the transaction, relying on the unique index on ReferenceNo as the safety net.	Simple and enough for the expected load; a database sequence needs extra schema work.	Two simultaneous submissions could produce the same number; the unique index rejects the second.	Replace with a SQL Server SEQUENCE (or retry once on conflict) when load or time allows.
TD-002	No login throttling or lockout, no password reset, no email verification.	The requirements do not need them, and building them was out of the M3 time budget.	Passwords can be guessed repeatedly; locked-out users need an administrator.	Add rate limiting or lockout first; move to ASP.NET Core Identity if more account features are approved.
TD-003	Own User table with cookie authentication instead of ASP.NET Core Identity.	Keeps one user model and one migration history, avoiding an extra package and integration risk.	Missing security features have to be built by hand.	Revisit with TD-002; the IPasswordHasher abstraction and policy names stay unchanged.
TD-004	Database-backed integration tests are skipped in CI (no	CI stayed simple and free; the in-memory pipeline	Some behaviour is only verified by	Add a SQL Server service container to the workflow or use

	SQL Server service); the performance test is run by hand	tests still run.	manual run.	a SQLite test provider.
TD-005	Layers are folders inside one web project, so the rule "controllers never use the DbContext" is enforced by code review, not by the compiler.	One project avoids reference errors and keeps the build simple for a three-person team.	A shortcut could slip through review.	Add architecture tests or split into class libraries if the code base grows.
TD-006	The CodeQL scan is report-only and not a required check.	There is no baseline yet, so a blocking scan could have blocked every PR for pre-existing findings.	A real finding could be merged.	Triage the first results, then make the scan a required check.
TD-007	The demo data seeder runs only in the Development environment; staging data is created by hand.	Avoids demo accounts with known passwords on a public site.	Staging set-up is manual and easy to get wrong.	Add a staging-safe seed that reads passwords from Application Settings.
TD-008	Requester feedback is in-app only.	External providers are outside the approved scope.	Requesters must open the system to see updates.	Revisit through change control with an adapter and a new risk entry if approved.

23) GITHUB GOVERNANCE, TEAM ACCOUNTABILITY & AI USAGE REGISTER

23.1) Repository governance evidence

- **Repository link**: https://github.com/AidanSm2103/SEN381_CivicConnect
- **Protected main branch**: Branch protection is enabled on main requiring a pull request before merging, with a minimum of 2 approving reviews and admin bypass disabled.
- **Required reviewer approvals**: Two approvals from members other than the author, no self-approval

23.2) Team working agreement

Enrique de Sousa: Problem Framing & Scope Lead
This role owns what CivicConnect is trying to solve and for whom, stakeholder needs, competing expectations, what falls within the scope and how schedule/cost/quality/security constraints interact with each other.

Michael-John Robinson : Requirements & Evidence Lead
This role involves turning stakeholder needs into testable commitments, writing FRs/NFRs with acceptance criteria and making sure every requirement traces back to a source.

Aidan Smith: Risk, Governance & AI Accountability Lead
This role is responsible for what could go wrong and how the team stays in control, risk identification/mitigation, thinking ahead to lifecycle concerns, repository governance and verifying AI-assisted work.

Collective responsibility is maintained through mandatory cross-review of at least one section not authored by the reviewer and all three members required as approvers on GitHub.

Communication Cadence: Brief in person meetings each day to align on progress and next steps, with the WhatsApp group used for quick questions, sharing updates and coordinating outside of those sessions.

23.3) AI Usage Register

Date	Team Member	AI Tool	Engineering task	AI Contribution	Verification Performed	Decision	Issues Found
2026/09/04	Aidan Smith	Claude	Creating the initial PED v1.0 document structure at the start of M1, based on the required outputs table in the Milestone 1 brief	Generated a section-by-section document with placeholder headings, prompts, and a proposed three-way work split	Cross-checked every generated section heading and sub-heading against the actual M1 brief's Required Outputs table	Adopted the skeleton structure as the working document framework with no content changes needed	None — structure matched the brief's required outputs exactly

24) ENGINEERING DECISION LOG

Decision ID	Context	Constraints	Alternatives	Decision	Rationale	Trade-offs	Risks	Evidence	Later Consequence
D-001	The team needed a consistent, defensible method to prioritise FRs and NFRs given a fixed schedule constraint, rather than ranking requirements informally	Schedule & Quality	(a) MoSCoW (Must/Should/Could/Won't), (b) stakeholder-vote-based ranking (c) High/Medium/Low tied directly to core business needs	Adopted option (c)	Directly traceable to the stakeholder needs and business value and to the security/accountability risks already identified	Less granular than MoSCow, so items may need review later on	Borderline requirements involve some subjective judgement; team members could disagree on edge cases	Section 3.1–3.3	To be revisited only via the formal Change Request/Impact Analysis process once the core system is stable
D-002	The team needed to select a	Team familiarity; maintainabi	(a) MERN , (b) ASP.NET	Adopted option (b)	ASP.NET aligned with the	Less access to the	Framework configurat	Technology-stack comparis	Future technology changes to be

	technology stack that could support the system's functional requirements while remaining maintainable	lity; security; integration			team's existing C# and SQL Server experience while providing more secure packages than npm	JavaScript/Node ecosystem than MERN; greater reliance on the Microsoft/.NET ecosystem	ion and architecture could introduce complexity	on and previous research done on both stacks	considered through the formal Change Request
D-003	A status change must give the requester feedback (FR-005) without weakening the audit trail (NFR-010)	Modular monolith (D-003); no external email/SMS in approved scope; Azure F1 hosting	(a) Synchronous in-process call in the same transaction; (b) asynchronous event + worker (Assignment 2 section 3 recommendation); (c) REST call between modules	Adopted option (a)	No external provider, so the async option's benefit does not apply; atomic with StatusHistory; no extra infrastructure; testable through DI	Feedback is in-app only; the status change waits for the feedback write (a local insert, negligible for NFR-001)	If email/SMS is added later, provider outages must not block status changes	Section 11; Assignment 2 section 3; Section 10	Adding external notifications requires change control, an adapter, a queue/worker and a new risk entry
D-004	Requests, users, history and comments need strong integrity and a complete audit trail (NFR-010)	Schedule, Cost/Resources, Security; team building in C# with EF Core + SQL Server	(a) Relational DB, synchronous transaction; (b) document DB; (c) relational DB with asynchronous audit write	Adopted option (a)	Structured, related data; atomic audit trail; SQL reporting; strong EF Core support	Schema changes need migrations; the audit write adds slightly to each transaction (acceptable under NFR-001)	SPOF / data loss	Section 8; A2 §2; A3 §3.4; AddCategoryLookup migration (PR #3).	A move to a non-relational store would need integrity rules rewritten in code
D-005	M2 selected ASP.NET Core Identity with policy-based authorisation. The project already owns its User and StaffCategory	Schedule, Security, Team capacity, Quality.	(a) Full ASP.NET Core Identity with IdentityDbContext; (b) cookie authentication + framework PasswordHasher + own User table; (c) a	Adopted option (b).	Meets every authentication requirement; keeps one user model and one migration	Lockout, password reset, email confirmation and two-factor authentication are	Login is not protected against repeated password guessing (R-006).	AuthService, IdentityPasswordHasher, AuthorizationPolicies and their tests	Any account-management feature beyond login/registration triggers a decision to adopt Identity or build it..

	Scope entities and has one migration owner.		third-party identity provider.		history; no extra package.	not provided.			
D-006	NFR-004 requires credentials to be hashed.	Security, login latency.	(a) keep the framework default; (b) raise the iteration count to the OWASP figure for this algorithm; (c) switch to another algorithm such as Argon2id.	Adopted option (b).	Meets current OWASP guidance without a new dependency; older hashes still verify and are flagged for re-hash.	Login takes roughly twice as long as with the default	Guidance changes over time.	OWASP Foundation; .NET Foundation; PasswordHasher Tests	Revisit if guidance changes or the hashing cost affects the performance results.
D-007	API/integration and E2E tests must run on every PR, but the GitHub runner has no SQL Server and the team wanted CI to stay free and simple.	(a) SQL Server service container in CI; (b) SQLite test provider; (c) EF Core in-memory provider behind WebApplicationFactory; (d) run integration tests manually only	Adopted (c): CivicConnectFactory boots the real app in the "Testing" environment and swaps only the DbContext to the in-memory provider.	Exercises the real routing, cookies, anti-forgery, DI, services, repositories and views on every PR with no infrastructure. Trade-off: no CHECK constraints, real rowversion or transactions in CI – covered by manual LocalDB checks.	Integration/CivicConnectFactory.cs (PR #26); 15 integration + 2 E2E cases in CI	R-007 / TD-004: move to (a) or (b) after M3 so database behaviour is also gated.	D-007	API/integration and E2E tests must run on every PR, but the GitHub runner has no SQL Server and the team wanted CI to stay free and simple.	(a) SQL Server service container in CI; (b) SQLite test provider; (c) EF Core in-memory provider behind WebApplicationFactory; (d) run integration tests manually only
D-008	Two PRs reached main with a broken build while CI was advisory (DEF-001, DEF-002).	(a) Keep CI advisory and rely on reviewers; (b) make build-test and dependency-audit required status checks with up-to-date branches	Adopted (b).	A machine check is cheaper and more reliable than reviewers spotting compile errors. Trade-off: PRs wait for CI and must be rebased/mer	Branch protection on main; PR #26 blocked until fixed	CodeQL to be added as a required check once its findings are triaged (TD-006).	D-008	Two PRs reached main with a broken build while CI was advisory (DEF-001, DEF-002).	(a) Keep CI advisory and rely on reviewers; (b) make build-test and dependency-audit required status checks with up-to-date branches

				ged with main first.					

25) PED QUALITY, DOCUMENT CONTROL & BASELINE SIGN-OFF

25.1) Document control

Version: v1.0			
Date	**Reviewer**	**Sections Reviewed**	**Outcome**
2026/10/16	Michael-John Robinson	Sections 6.1, 4.1, 14, 16, 17, 18 , 20	Approved
2026/10/16	Aidan Smith	Sections 6.1, 14, 15, 19, 21	Approved
2026/10/16	Enrique de Sousa	Sections 13, 17, 18, 25	Approved
2026/10/16	Entire team	Full document	Approved sign-off

25.2) Baseline sign-off

Project	CivicConnect
Baseline Type	M3 - Controlled Construction, Integration, Quality & Release Readiness
Version	v3.0
Date	2026/10/16
Release candidate identified	YES
M2 baseline conformance and controlled changes checked	YES
Core scope integrated and working	YES
Repository/governance controls checked	YES

Automated tests and CI quality gate evidenced	YES
Black-box, integration and E2E validation evidenced	YES
Static and security checks evidenced	YES
Performance test documented and interpreted	YES
Staging/test environment demonstrated	YES
RTM, registers and ADRs updated	YES
Release Evidence Summary and residual risk completed	YES
Outcome	CONDITIONALLY ACCEPTED

26) REFERENCES

Pilliang, M. and Munawar, M., 2022. Risk management in software development projects: A systematic literature review. *Khazanah Informatika: Jurnal Ilmu Komputer dan Informatika*, *8*(2).

Verdecchia, R., Kruchten, P., Lago, P. and Malavolta, I., 2021. Building and evaluating a theory of architectural technical debt in software-intensive systems. *Journal of Systems and Software*, *176*, p.110925.